МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 3
по дисциплине «Программирование»

Тема: «Изучение механизмов наследования в языке C++»

Выполнил:
студент гр. 550503
Минкевич А. С.

Проверил:
ст. преподаватель
Ковальчук А. М.

Минск, 2026

ВВЕДЕНИЕ

0.0.1 Цель работы: изучить принципы наследования в языке C++ (простое и множественное наследование), получить практические навыки проектирования производных классов, вызова конструкторов базовых классов и работы с динамической памятью в иерархии классов.

Задание на лабораторную работу (вариант 5): Разработать класс `Fakultet`, порождающий класс `Student`.

– Класс `Fakultet` включает компонент данных: наименование факультета.

– Класс `Student` включает компоненты данных: Ф.И.О. студента, год рождения, результаты сдачи последней сессии.

Необходимо реализовать конструкторы (в том числе конструктор копирования), деструкторы, методы доступа и изменения полей, а также методы ввода и вывода данных.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Наследование и модификаторы доступа

1.1.1 Наследование — это механизм получения нового класса (производного) из существующего (базового). Производный класс получает данные и методы базового класса и может дополнять их собственными. При простом наследовании класс порождается одним базовым классом, при множественном — несколькими.

Объявление производного класса имеет вид: `class Student : public Fakultet { ... };`

При открытом наследовании (`public`) права доступа к элементам базового класса сохраняются: `public` остаётся `public`, `protected` остаётся `protected`, а `private`-элементы базового класса в производном классе недоступны напрямую. Права доступа при наследовании могут быть только ограничены, но не расширены.

1.2 Конструкторы и деструкторы в иерархии классов

1.2.1 Если базовый класс имеет конструктор с параметрами, то производный класс обязан иметь конструктор, который передаёт нужные аргументы в конструктор базового класса через список инициализации: `Student(...) : Fakultet(faculty), ... { ... }`

Конструкторы вызываются в порядке наследования: сначала конструктор базового класса, затем конструктор производного. Деструкторы вызываются в обратном порядке. Если объект производного класса удаляется через указатель на базовый класс, деструктор базового класса должен быть виртуальным, иначе деструктор производного класса не будет вызван и произойдёт утечка памяти.

1.3 Виртуальные функции и переопределение

1.3.1 Виртуальная функция объявляется в базовом классе с ключевым словом `virtual` и переопределяется в производных классах. Вызываемая версия определяется во время выполнения программы по типу объекта. Спецификатор `override` позволяет компилятору проверить, что функция действительно переопределяет виртуальную функцию базового класса. Вызов с полным именем (`Fakultet::print()`) отключает виртуальный механизм и вызывает версию конкретного класса.

2 ХОД РАБОТЫ

2.1 Проектирование классов

2.1.1 Разработана иерархия из двух классов. Базовый класс `Fakultet` содержит закрытое поле `facultyName` (наименование факультета), методы `getFacultyName()` и `setFacultyName()`, виртуальный метод `print()` и виртуальный деструктор.

Производный класс `Student` наследуется от `Fakultet` с модификатором `public` и содержит закрытые поля:

– `fullName` — Ф.И.О. студента;

– `birthYear` — год рождения;

– `examsCount` — количество экзаменов в последней сессии;

– `results` — указатель на динамический массив оценок последней сессии.

2.2 Реализация методов класса Student

2.2.1 Для класса `Student` реализованы:

1 Конструктор с параметрами `Student(faculty, exams)`, который передаёт наименование факультета в конструктор базового класса, выделяет память под массив оценок и обнуляет его.

2 Конструктор с параметрами, принимающий все поля сразу, включая массив оценок.

3 Конструктор копирования, выполняющий глубокое копирование: копируется часть базового класса, а для массива `results` выделяется собственная память. Это предотвращает ошибку двойного освобождения памяти.

4 Деструктор, освобождающий память с помощью `delete[]`.

5 Оператор присваивания `operator=` с проверкой на самоприсваивание, освобождением старого массива и выделением нового.

6 Методы доступа и изменения всех полей: `getFullName()`, `getBirthYear()`, `getExamsCount()`, `getMark()`, `setFullName()`, `setBirthYear()`, `setMark()`, `setResults()`. В методах работы с оценками выполняется проверка выхода индекса за границы массива.

7 Метод `average()`, вычисляющий средний балл за сессию с защитой от деления на ноль.

8 Метод `input()` для ввода данных с клавиатуры.

9 Метод `print()`, переопределяющий виртуальный метод базового класса: сначала вызывается `Fakultet::print()` для вывода наименования факультета, затем выводятся данные студента.

2.3 Организация функции main

2.3.1 В функции `main` пользователь вводит наименование факультета, количество студентов и количество экзаменов. Студенты хранятся в динамическом массиве указателей `Student**`, каждый объект создаётся оператором `new`. После ввода выводится список студентов, затем создаётся копия первого студента для проверки конструктора копирования. Перед завершением итерации цикла вся выделенная память освобождается.

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.0.1 Программа была протестирована на двух студентах с двумя экзаменами в сессии. Также с помощью инструмента AddressSanitizer проверено отсутствие утечек памяти и ошибок работы с памятью. Ниже представлен результат вывода программы.

Пример вывода консоли при выполнении программы

```
Enter faculty name: FIT
Enter number of students: 2
Enter number of exams in the last session: 2

Student 1:
Enter full name: Ivanov Ivan Ivanovich
Enter birth year: 2005
Enter mark for exam 1: 9
Enter mark for exam 2: 8

Student 2:
Enter full name: Petrov Petr
Enter birth year: 2006
Enter mark for exam 1: 7
Enter mark for exam 2: 10

Students list:

Faculty: FIT
Full name: Ivanov Ivan Ivanovich
Birth year: 2005
Session results: 9 8
Average mark: 8.5

Faculty: FIT
Full name: Petrov Petr
Birth year: 2006
Session results: 7 10
Average mark: 8.5

Copy of the first student:
Faculty: FIT
Full name: Ivanov Ivan Ivanovich
Birth year: 2005
Session results: 9 8
Average mark: 8.5
```

```
Continue? (y/n): n
```

ЗАКЛЮЧЕНИЕ

3.0.2 В ходе выполнения лабораторной работы были изучены принципы наследования в языке C++. Была спроектирована и реализована иерархия классов `Fakultet` и `Student`, в которой производный класс расширяет данные и функциональность базового.

Практически закреплены навыки передачи параметров в конструктор базового класса через список инициализации, использования виртуального деструктора и переопределения виртуальных методов со спецификатором `override`. Для класса `Student`, владеющего динамическим массивом оценок, реализованы конструктор копирования, деструктор и оператор присваивания, что предотвращает утечки памяти и двойное освобождение. Все реализованные методы работают корректно и соответствуют заданию.